

Wi-Fi Password Cracking — Demonstration Report

Project 20 · One Setup, Three Attacks, on Captured pcaps

Students: Mohammad Saifuzzaman (2105088)

Sayaad Muzahid Masfi (2105066)

Section / Group: B1 — Group 3

Course: CSE 406 Computer Security

All data and figures in this report were produced by our own tool (`wpacrack`, standard library only) on host `acer`, Python 3.12.3, on 2026-09-17 00:04. Regenerate with `python report/make_data.py`.

1 What we demonstrate

We built one offline tool that runs **three distinct password-cracking attacks** against **captured pcap files**, plus a defense analyzer. Everything is *file-in / verdict-out*: no radio, no packet injection, no live traffic. The three attacks share a single cryptographic core, so this is genuinely “multiple attacks on one setup” rather than three separate programs.

Attack	What differs	Verifier used to confirm a guess
1. Handshake MIC	candidate = wordlist	derive PTK, recompute EAPOL MIC, compare
2. PMKID	needs only Message 1	recompute PMKID = HMAC-SHA1(PMK,...)
3. Mask / brute	candidate = pattern	same verifiers, candidates from a <code>?d?l?u?s</code> mask

2 System architecture (the “one setup”)

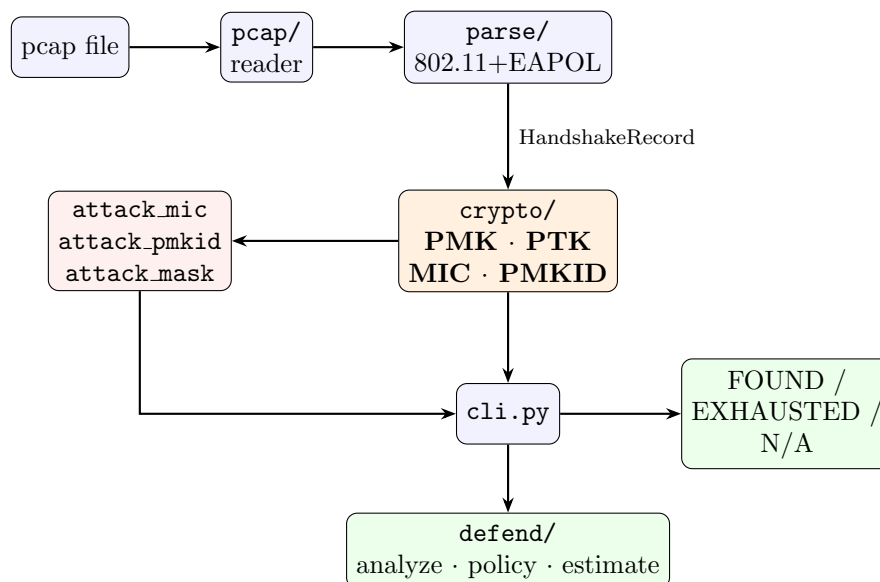


Figure 1: One toolchain. Only the attack module changes; the crypto core, parser, and CLI are shared. This is the “one setup” the supervisor asked for.

3 The WPA2 4-way handshake and where we attack

The attacker never needs the network key on the wire — it is never sent. Instead we capture the handshake and verify guesses *offline*. Attack 1 needs the ANonce (Message 1) and the SNonce + MIC (Message 2); Attack 2 needs only the PMKID carried in Message 1.

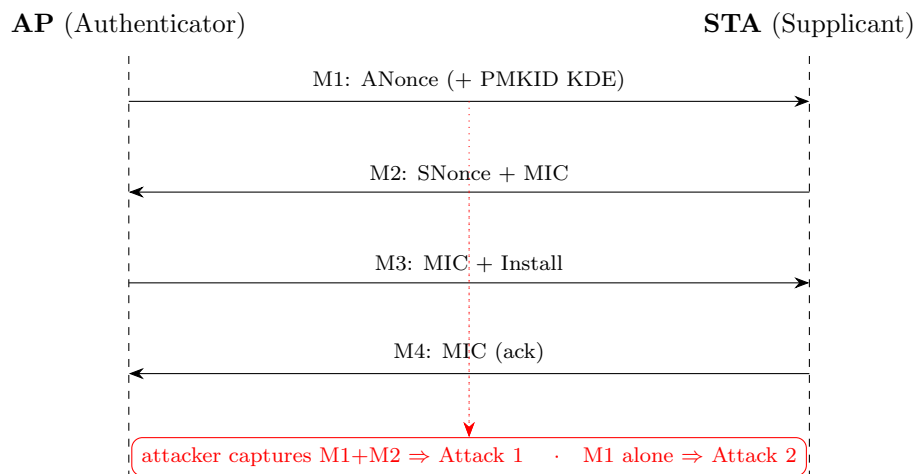


Figure 2: We recover the passphrase from passively captured frames. The candidate check is $\text{PMK} = \text{PBKDF2}(\text{passphrase}, \text{SSID})$, then either the MIC (Attack 1) or the PMKID (Attack 2) is recomputed and compared.

4 Live demo script

Run from the repository root. Each command is one slide; the expected verdict is what the audience should see.

1. **Generate the target captures** (fabricated in software — nothing illegal):

```
$ python3 -m wpacrack.sim
```

2. **Attack 1 — crack from a full handshake:**

```
$ python3 -m wpacrack mic --pcap fixtures/hs_ok.pcap \
  --wordlist wl/demo.txt --ssid demoAP
=> [MIC] FOUND passphrase='password123'
```

3. **Attack 2 — crack from only a PMKID (no full handshake):**

```
$ python3 -m wpacrack pmkid --pcap fixtures/pmkid.ok.pcap \
  --wordlist wl/demo.txt --ssid demoAP
=> [PMKID] FOUND passphrase='password123'
```

4. **Attack 3 — mask brute force a 6-digit PIN, time-boxed:**

```
$ python3 -m wpacrack mask --pcap fixtures/hs_short.pcap \
  --mask '?d?d?d?d?d?' --timebox 60 --ssid demoAP
=> [MASK] FOUND passphrase='001234'
```

5. Negative control — prove no false positives:

```
$ python3 -m wpacli mic --pcap fixtures/hs-ok.pcap \
  --wordlist wl/negative.txt --ssid demoAP
⇒ [MIC] EXHAUSTED (searched, no match)
```

6. Defense — WPA3-SAE has no offline target:

```
$ python3 -m wpacli analyze --pcap fixtures/wpa3.sae.pcap
⇒ [ANALYZE] OK-SAE no offline PMK/PMKID target
```

7. Real-world proof — crack a public capture:

```
$ python3 -m wpacli mic --pcap fixtures/public/wpa-Induction.pcap \
  --wordlist wl.txt
⇒ [MIC] FOUND passphrase='Induction' (ssid=Coherer)
```

5 Results

Table 1 is the actual output of `python3 -m wpacli report`, which re-runs every scenario and asserts each verdict (the demo is also a self-test).

Scenario	Cond.	Result	Password	Tried	Time	Rate/s
Attack 1: Handshake MIC	dict hit	FOUND	password123	3	0.01	255
Attack 2: PMKID	dict hit	FOUND	password123	3	0.01	237
Attack 3: Mask brute	6-digit PIN	FOUND	001234	1,235	5.47	226
Attack 1: negative control	dict miss	EXHAUSTED	–	5	0.02	233

Table 1: Demonstration results (this machine). FOUND = verified; EXHAUSTED = searched with no match (no false positive).

6 Performance: why the implementation matters

WPA2 forces PBKDF2-HMAC-SHA1 with 4096 iterations per candidate — this is the whole cost of the attack. Using Python’s C-backed `hashlib.pbkdf2_hmac` instead of a hand-rolled loop is our one big performance lever.

This measured rate (162 cand/s) is exactly what drives the crack-time estimates in the defense section — the attack cost model and the defense advice use the same number.

7 Defense

Analyzer. `defend/analyze` reads the RSN AKM suite from a beacon and gives a verdict: WPA2-PSK ⇒ **crackable**; WPA3 transition mode ⇒ **WPA2 fallback still crackable**; WPA3-SAE-only ⇒ **no offline target**. This is why offline dictionary/PMKID attacks fail against SAE (Dragonfly): there is no PMK-derived value exposed to capture.

Strength. Figure 4 and Table 2 use our measured rate to show what survives an offline attack. Short numeric or single-charset passphrases fall quickly; long mixed passphrases are infeasible.

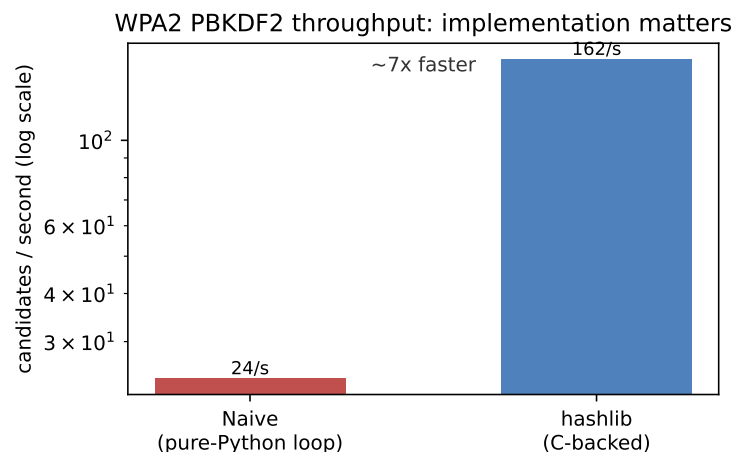


Figure 3: Measured candidate throughput: 162 cand/s with `hashlib` versus 24.1 cand/s with a naive pure-Python loop ($7\times$ faster) on host `acer`.

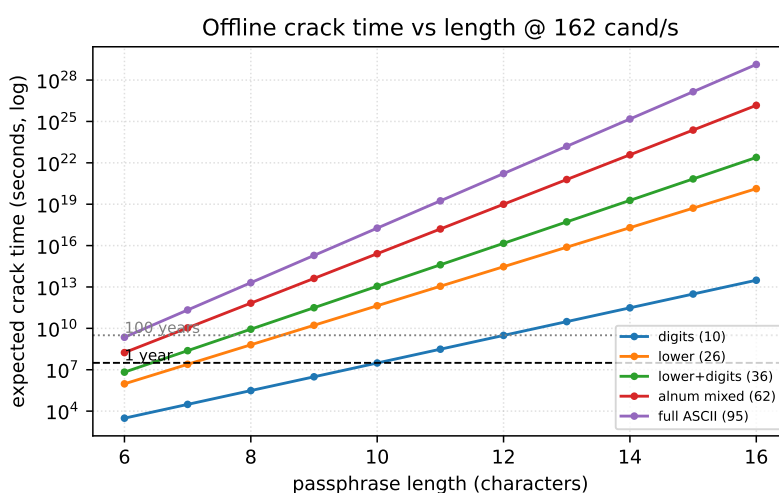


Figure 4: Expected offline crack time vs passphrase length, per character set, at 162 cand/s. Dashed line = 1 year.

Recommendation. Use a long (≥ 15 char) mixed passphrase; prefer WPA3-SAE-only and disable WPA2 transition mode; keep SAE implementations patched (Dragonblood, 2020).

8 Correctness and honesty of the demo

- **Known-answer vector:** our PMK matches the canonical IEEE 802.11i value `f42c6f...a12e` for (password, IEEE).
- **Real capture:** the full PMK $\rightarrow$ PTK $\rightarrow$ MIC path recovers `Induction` from the public `wpa-Induction.pcap` (SSID `Coherer`); wrong guesses fail. This proves the code matches real 802.11, not just our own generator.
- **Automated tests:** `pytest` runs 16 tests green (KATs, QoS frames, appended-FCS handling, missing-SSID, negative control, real capture).
- **No false positives:** the negative-control scenario searches a wrong wordlist and returns

Charset	Len	Keyspace	Expected time	Verdict
d	6	1.0×10^6	51.3m	WEAK
d	8	1.0×10^8	3.6d	WEAK
l	8	2.1×10^{11}	20.4y	OK
a	8	6.6×10^{15}	647405.3y	OK
a	12	5.4×10^{23}	$> 10^6$ yr	OK
a	16	4.4×10^{31}	$> 10^6$ yr	OK

Table 2: Crack-time estimates from `defend/estimate` at the measured rate (d=digits, l=lowercase, a=full ASCII).

EXHAUSTED, never a bogus FOUND.

- **Scope/ethics guard:** `tools/ethics.guard.py` fails the build if any socket / injection / raw-transmit code appears in the tool.

A Appendix: source layout

<code>wpacrack/crypto/</code>	PMK, PRF-512/PTK, MIC (v1/v2), PMKID — shared core
<code>wpacrack/pcap/</code>	pcap + pcapng file reader
<code>wpacrack/parse/</code>	radiotap/802.11/EAPOL deframe, RSN IE, SSID → HandshakeRecord
<code>wpacrack/attack_mic/</code>	Attack 1 driver
<code>wpacrack/attack_pmkid/</code>	Attack 2 driver
<code>wpacrack/attack_mask/</code>	Attack 3 driver (mask + time-box)
<code>wpacrack/defend/</code>	<code>analyze.py</code> , <code>policy.py</code> , <code>estimate.py</code>
<code>wpacrack/sim/</code>	synthetic handshake/pcap generator
<code>wpacrack/cli.py</code>	unified CLI: <code>mic pmkid mask analyze policy estimate report</code>
<code>tests/</code>	16 pytest cases (unit + integration + real capture)
